

Independent Study Course (NST3902)

# Incremental Development of Machine Learning Systems for Single-Frame Football Field Localisation

30.04.2026

Nguyen Hoang The Kiet

NUS College, National University of Singapore

## Contents

|                                                                         |           |
|-------------------------------------------------------------------------|-----------|
| <b>1. Introduction</b>                                                  | <b>2</b>  |
| <b>2. An Incremental Development Pipeline</b>                           | <b>3</b>  |
| <b>3. Task Description</b>                                              | <b>5</b>  |
| <b>4. Related Works</b>                                                 | <b>6</b>  |
| <b>5. Methodology</b>                                                   | <b>8</b>  |
| 5.1. Training procedure .....                                           | 8         |
| 5.2. Inference procedure .....                                          | 10        |
| <b>6. Experiments</b>                                                   | <b>10</b> |
| <b>7. Commentary and Future plans</b>                                   | <b>12</b> |
| <b>Bibliography</b>                                                     | <b>13</b> |
| <b>A. Project Github repository</b>                                     | <b>14</b> |
| <b>B. Participation in the SynLoc Spiideo Challenge, SoccerNet 2026</b> | <b>14</b> |
| <b>C. Live reports on Wandb</b>                                         | <b>14</b> |

## List of Figures

|           |                                                                                                                                                                                       |    |
|-----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| Figure 1  | Image from the Camera Calibration [1] task (left) versus the Spiddeo SymLoc dataset (right)                                                                                           | 2  |
| Figure 2  | The project's current development pipeline                                                                                                                                            | 3  |
| Figure 3  | Input image and keypoint annotations                                                                                                                                                  | 5  |
| Figure 4  | LocSim is performed on world coordinates (right)                                                                                                                                      | 6  |
| Figure 5  | Full pipeline of the LocationDetector module. The red parts indicate trainable components, whilst other components are non-trainable and are only available upon evaluation.          | 8  |
| Figure 6  | Original heatmap generated from heatmap_unet; Gaussianised heatmap with $\lambda = 0.1$ ; Gaussianised heatmap with top- $k$ sampling and $\lambda = 1$ ( <i>from left to right</i> ) | 9  |
| Figure 7  | Gaussianised heatmap with $\lambda = 0.5$ during training. The top- $k$ pixels are concentrated around a single keypoint.                                                             | 9  |
| Figure 8  | Loss charts after 32 epochs with a limited dataset of 256 images, resized at dimensions $720 \times 405$ .                                                                            | 10 |
| Figure 9  | False positive pixels on the boundary between the field area and the spectators' seating area.                                                                                        | 11 |
| Figure 10 | Diagnostic of prediction vs ground truth annotations on test image 000977.jpg                                                                                                         | 12 |

# 1. Introduction

The previous Independent Study Course (ISC-1) established a three-step machine learning pipeline method for the field localisation task, i.e. to determine the region and boundaries of a football field from single-view frames. However, during the actual development stage, many technical difficulties were soon realised, steering the project away from original methodological research to repetitive pipeline engineering tasks. Acknowledging the challenges mentioned above, this project aims to evaluate the efficiency of pipeline automation measures in the context of single-frame football field localisation.

Specifically, the proposed benefit for this ISC project (ISC-2) is two-fold:

- To automate certain parts of the development of the existing project. The project shall analyse the repetitive tasks observed in the process of training and deploying ML models, then automate them through an ML library. This process is expected to speed up the development process and utilise computational resources economically.
- To adapt the current project for participation in a new official machine learning challenge. This report introduces a new field localisation task, Spiideo SoccerNet SynLoc 2026 [2], a live challenge introduced by SoccerNet 2026. This new task is a continuation of previous tasks on field localisation and camera calibration in SoccerNet 2025, with two main differences of the input feed:
  - (a) *they are single- and wide-framed, covering a majority of the field;* and
  - (b) *they are subject to radial distortion.*

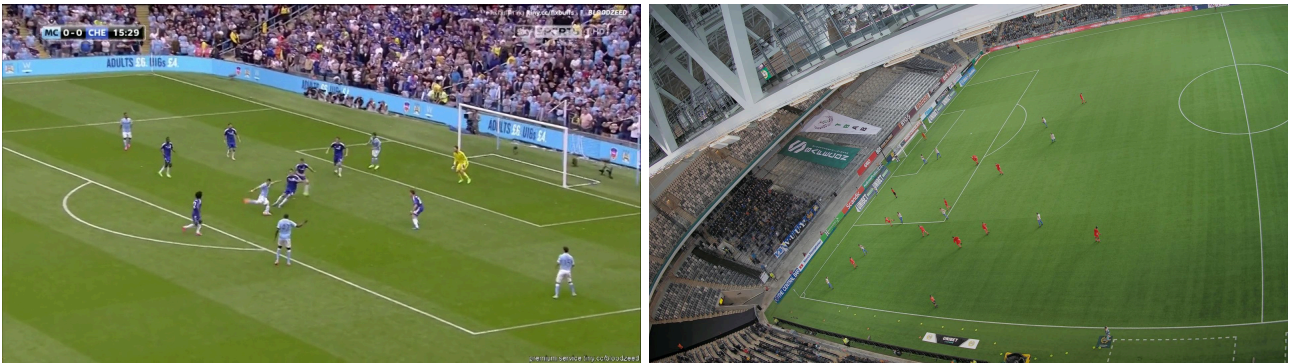


Figure 1: Image from the Camera Calibration [1] task (left) versus the Spiideo SymLoc dataset (right)

This report serves as a reflection after establishing the needed infrastructure for scaling up the project, and the project's initial participation in the Spiideo SynLoc challenge [2], with qualitative methodology reviews and actionable future plans moving forward.



## 2. An Incremental Development Pipeline

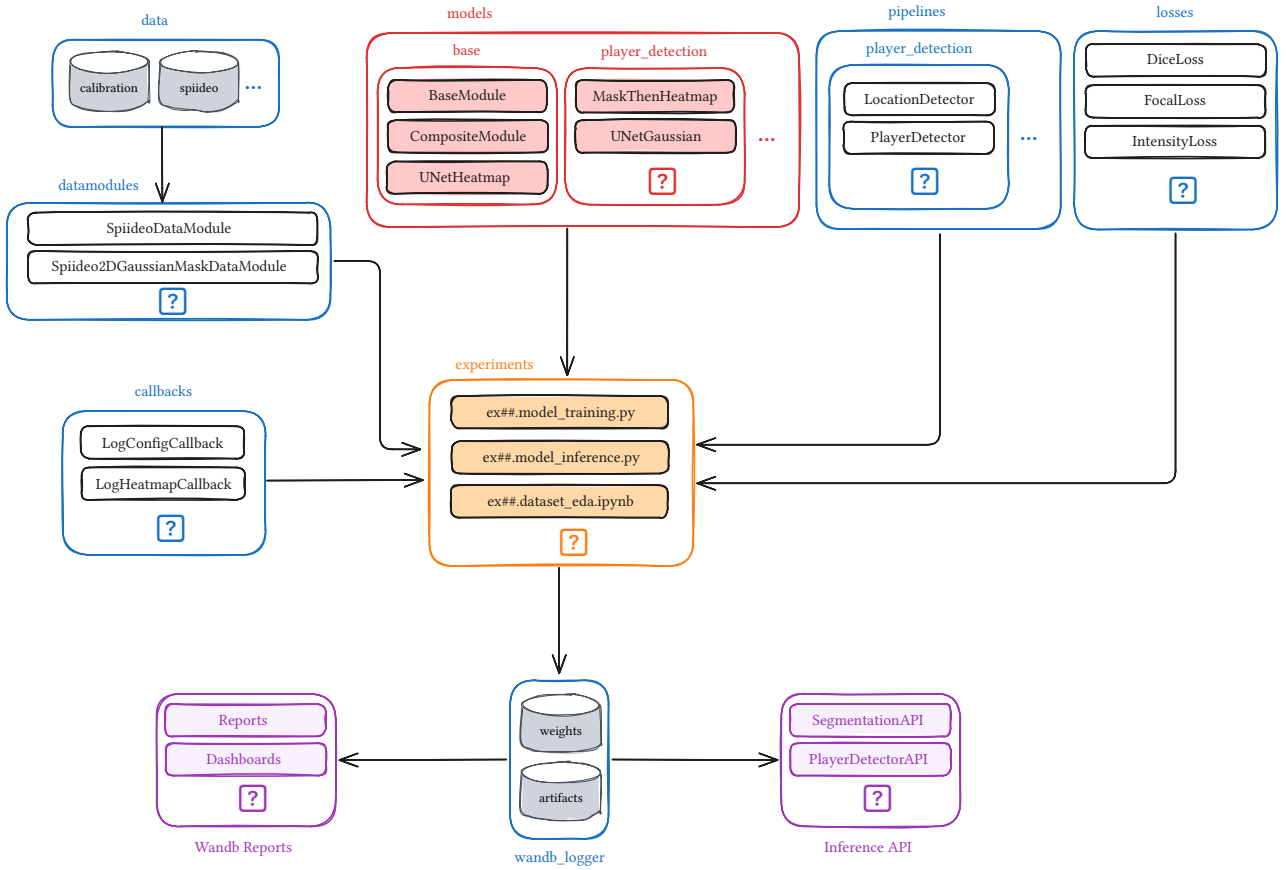


Figure 2: The project's current development pipeline

As the project grows with different tasks and diverse sources of datasets, the pure notebook approach in ISC-1 proves to be insufficient. Whilst the project already has separate modules for calling certain dataset (original or augmented), it still lacks a reproducible training pipeline. Moreover, each model requires multiple, possibly overlapping, steps of input preprocessing, therefore the preprocessing scripts must also be separated as a designed class for reusability.

To refactor the project codebase, we follow a selection of best practices from Imperial College London's *Deep Learning Best Practices* [3]. Due to time constraints and because of the already sophisticated codebase from the first ISC, we only adapt certain suggestions, whilst skipping on some libraries such as *Hydra* or *Einops*. These are the possible points for improvement moving forward when another code refactoring is needed.

After careful consideration, PyTorch Lightning [4] is chosen because of its modularity and extendability onto the cloud.

- **Modularity.** Each “module” in the project can be expressed as a `pl.LightningModule` object. The modules are equipped with prewritten scripts to interact with `pl.Trainers` (for training) and loggers, whilst being flexible as the developer can customise the module via one of its functions in the `pl.LightningModule` API.

| Tools                                        | Usage | Intended/Alternative use                                          |
|----------------------------------------------|-------|-------------------------------------------------------------------|
| <i>Wandb</i> , metrics and artifacts logging | ●     |                                                                   |
| <i>Hydra</i> , manage configurations         | ●     | <b>Alternative:</b> Uses <code>config.yaml</code> for experiments |
| <i>Pytorch Lightning</i> , training models   | ●     | Supports multicore GPU use, intergratability with cloud GPU usage |
| <i>Einops</i> , tensor multiplcation         | ●     | Interpretable tensor manipulation                                 |
| Training on GPU                              | ●     | Supported via <i>Pytorch Lightning</i>                            |
| Project template                             | ●     |                                                                   |
| ●: adapted, ●: yet to be adapted             |       |                                                                   |

Table 1: Personal reflection summary on the six ‘Deep Learning best practices’ [3] by Imperial College London

- **Compability with on-device and on-demand GPUs.** Training experiments are automatically orchestrated via `pl.Trainer`. It handles the usage of GPUs on the background and communicates with the `DataModules` to load batched input data and with the loggers to output the necessary artifacts and weights.

It is important that the project needs to be flexible because the datasets, models and pipelines are prone to methodological and technological changes in the future. We shall adapt ICL’s [3] project structure, with some add-ons to the modules to fit the needs of the project.

Our project consists of isolated modules:

- **data:** contains raw data, including images (in `.jpg` format) and JSON annotation files.
- **datamodules:** contains `pl.LightningDataModules` that are responsible of reading input images from the **data** folder, and perform any transformation on the images before feeding for model training.
- **models:** contains trainable `pl.LightningModules`.
- **pipeline:** contains non-trainable `pl.LightningModules`, only for evaluation. These are mainly composition modules that connects trainable models with non-trainable layers to perform a full prediction pipeline.
- **losses:** contains non-trainable `pl.LightningModules` to determine the loss function between  $y_{\text{pred}}$  and  $y_{\text{true}}$ .
- **callbacks:** contains callback functions that are utilised by the logger to log model artifacts, which will become useful for later model diagnostics.

These modules are then orchestrated by experiment scripts in the **experiments** folder. These are where the training happens, with configuration parameters determined within each individual experiment via the `config.yaml` file.

### 3. Task Description



Figure 3: Input image and keypoint annotations

The project has participated in the SoccerNet 2026 competition on the “*Single Frame World Coordinate Athlete Detection and Localisation*” task [2]. The task consists of two main parts; (a) detecting players on the pitch through a wide-view camera, and (b) localising the players, i.e. locating the world coordinates of each player on the field.

The annotations follow the COCO annotation model [5], where the JSON category object only contain a single category ‘person’. Each image has an associated camera matrix  $\begin{bmatrix} \mathbf{R} & \vec{t} \end{bmatrix}$ , a distortion polynomial  $p_{\text{dist}}$  and a undistortion polynomial  $p_{\text{undist}}$ . A point  $(x, y)$  on the image coordinate system can be transformed into the world coordinate system as follows (see `datamodules/spiideo/coco.py`):

- **Undistort the coordinate grid:**  $p_{\text{undist}}$  is a transformation on the normalised radius component of each pixel. A point  $(r, \theta)$  in the polar coordinate system is transformed to  $(\tan(p_{\text{undist}}(r)), \theta)$  after undistortion. Therefore, let  $(u, v) = \frac{1}{w} \left( x - \frac{w}{2}, y - \frac{h}{2} \right)$  be the normalised cartesian coordinates of the pixel at image coordinate  $(x, y)$ , then the new radius component of the transformed point is

$$r' = \tan \left( p_{\text{undist}} \left( \sqrt{u^2 + v^2} \right) \right)$$

The new point  $(x', y')$  is constructed accordingly.

- **Homographic inverse transformation:** by setting the  $z$ -coordinate in both the image and the world coordinates to 0, the third column in the camera matrix can be removed, leaving us with the

homographic transformation  $\mathbf{H} = [\hat{\mathbf{r}}_1 \ \hat{\mathbf{r}}_2 \ \vec{t}]$  that maps a world-coordinated point into a image-coordinated pixel. The world coordinates are therefore

$$\lambda \begin{bmatrix} x_{\text{world}} \\ y_{\text{world}} \\ 1 \end{bmatrix} = \mathbf{H}^{-1} \cdot \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix}$$

Each player is annotated with two keypoints: one at the point of contact between the player’s feet and the field, the other that the player’s pelvis. For the purpose of the proposed methodology, we are only interested in the first type of points.

The metric to evaluate the prediction of players’ position is the LocSim metric [2], performed on **world coordinates**, where a ground truth point  $\vec{G}$  and the prediction point  $\vec{P}$  have the LocSim of

$$\text{LocSim}(\vec{P}, \vec{G}) = e^{(\ln 0.05) \|\vec{P} - \vec{G}\|^2}$$

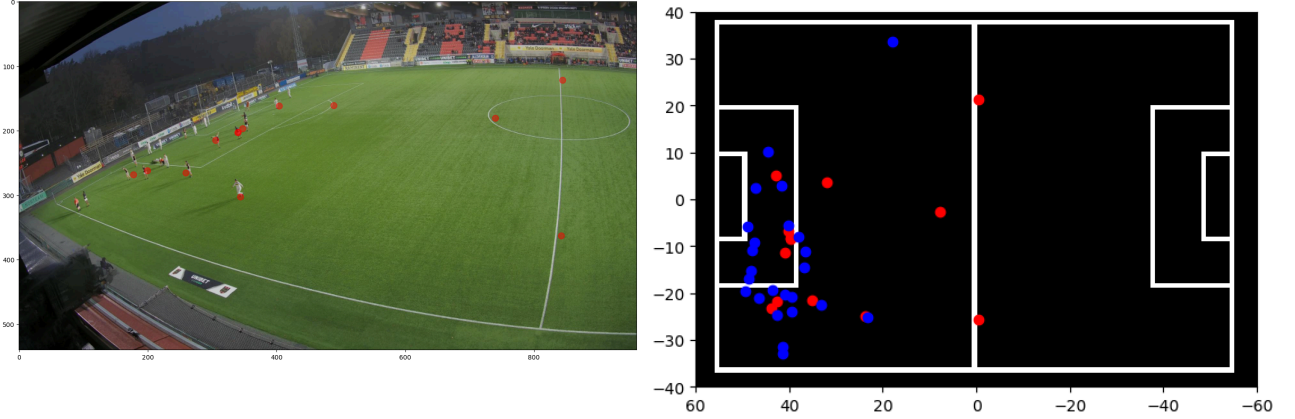


Figure 4: LocSim is performed on world coordinates (right)

## 4. Related Works

*U-Net as a heatmap generator*, Ronnerberger et al, 2015. [6]

U-Net is a convolutional-style neural network, with a series of down-convolutional layers followed by up-convolutional layers, and ‘copy-and-crop’ layers that propagate pre-down-sampled features to the up-sampling layers. The network architecture is known to be robust with datasets of a limited size, with many application in segmentation or heatmap generation. In ISC-1, we used U-Net to segmentate field lines in broadcast images of football matches with decent results, and we also explored using U-Net to detect keypoints (intersection of field lines), however with poorer results. ISC-2 will mainly focus on improving U-Net’s capability to detect keypoints using heatmap generation.

*Numerical Coordinate Regression with Convolutional Neural Networks*, Nibali et al, 2018 [7].

Nibali et al. proposes a non-parametric differentiable spatial to numerical transform (DSNT) layer to regress a 2D heatmap into two coordinates  $(x^*, y^*)$  that resembles the peak point of the heatmap by applying element-wise matrix multiplication of the predetermined kernels  $\mathbf{X}, \mathbf{Y}$  to the heatmap  $\mathbf{Z}$ . Specifically, with kernels  $\mathbf{X}, \mathbf{Y}$  predetermined,

$$x^* = \|\mathbf{Z} \odot \mathbf{X}\|_F, \quad y^* = \|\mathbf{Z} \odot \mathbf{Y}\|_F$$

where  $\odot$  is the element-wise matrix multiplication operator, and  $\|\cdot\|_F$  is the matrix Frobenius norm. As the layer consists of differentiable matrix operations, DSNT effectively changes the problem from heatmap regression to coordinate regression, which we can utilise simple loss functions such as L1-Loss, L2-Loss, etc., though a secondary loss function on the heatmap is still needed.

***Focal Loss for Dense Object Detection, Lin et al, 2018 [8]***

When regressing heatmaps, we notice that there is a class imbalance, with a bias towards black pixels. Typical loss functions such as  $L_2$  or BinaryCrossEntropy penalise wrongly predicted white pixels as much as wrongly predicted black pixels, therefore will just incentivise the model to produce a blank heatmap of all black pixels. Lin et al’s [8] FocalLoss aims to mitigate this class imbalance issue. Recall that the BinaryCrossEntropy loss is defined as

$$\text{BCE} = - \sum_{t \in T} y_t \log \hat{y}_t$$

FocalLoss introduces the term  $(1 - \hat{y}_t)^\gamma$  to decrease the weightage of the loss term at class  $t$  when the predicted probability  $\hat{y}_t$  gets close to 1.

$$\text{FL} = - \sum_{t \in T} y_t \cdot \alpha_t (1 - \hat{y}_t)^\gamma \log \hat{y}_t$$

***Heatmap Distribution Matching for Human Pose Estimation, Liu et al, 2018 [9].***

Liu et al proposes improvements on heatmap regression methods using the framework of *heatmap distribution matching* by setting a loss function on the predicted heatmap and the actual Gaussian-smoothed heatmap. Pixel-wise aggregate losses such as the Mean Squared Error (MSE) do not always guide the heatmap towards localising the correct position, but can instead generate heatmaps that mislead the chosen position after argmax. The paper uses the ‘Earth Mover’s Distance’ to compare the predicted and the annotated heatmap in the distribution sense, analogously to other distribution distances such as the Kullback-Leibler distance. Although we will not study this paper in detail, this gives us decent justifications on *heatmap matching* approaches.

## 5. Methodology

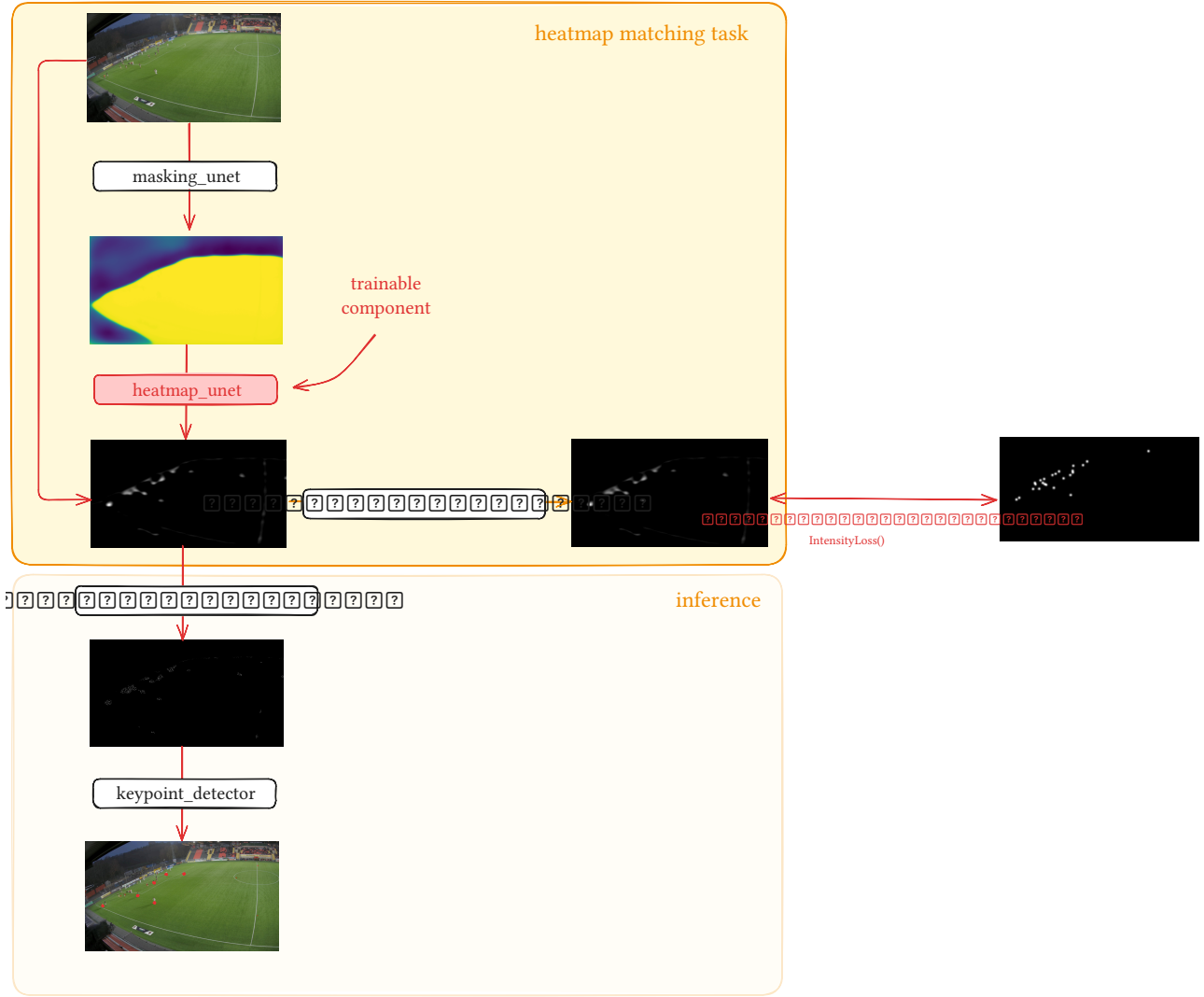


Figure 5: Full pipeline of the LocationDetector module. The red parts indicate trainable components, whilst other components are non-trainable and are only available upon evaluation.

We define the PlayerDetector model as a composition of submodules with two main stages. The first stage is to generate a Gaussian heatmap based on the location of the players. The model for this first stage is subject to a heatmap matching task and is available for training. The second stage is to convert the Gaussian heatmap into a collection of keypoints, which we will utilise the Bayesian Gaussian Mixture Model (BGMM) algorithm to compute the components' centre. This module is non-trainable.

### 5.1. Training procedure

For the heatmap generation model, we first use the **masking\_unet** module to segment the inner section of the field from the outer sections (such as the audience's area, the sky, etc.). To train **masking\_unet** we utilise the project during ISC-1, where the calibration dataset [10] is augmented so that the area inside the field lines are captured. The images are annotated such that a point in the image coordinates system  $(x, y)$  is deemed to lie inside the playing field area if its world coordinates



equivalent lies in the playing area, which is bounded by a rectangular area. The conversion can be done in batches, i.e.

$$\begin{bmatrix} x_1^{\text{world}} & x_2^{\text{world}} & \dots & x_n^{\text{world}} \\ y_1^{\text{world}} & y_2^{\text{world}} & \dots & y_n^{\text{world}} \\ \lambda_1 & \lambda_2 & \dots & \lambda_n \end{bmatrix} = \mathbf{H}^{-1} \cdot \begin{bmatrix} x_1 & x_2 & \dots & x_n \\ y_1 & y_2 & \dots & y_n \\ 1 & 1 & \dots & 1 \end{bmatrix}$$

where the construction of the homographic transformation  $\mathbf{H}$  is described in details in ISC-1. A point  $(x, y)$  is in the field iff.  $|x^{\text{world}}| \leq W/2$  and  $|y^{\text{world}}| \leq H/2$ , where  $W \times H$  are the predetermined dimensions of the field. For this project, we choose  $W = 110$  and  $H = 55$ .

heatmap\_unet is a generic U-Net that accepts a multi-channel image and produces a single-channel heatmap. To ensure that the heatmap is incentivised to predict actual players' locations, we also introduce another gaussianiser layer. The key observation here is that the heatmap does not represent a true Gaussian heatmap; the gaussianiser is there to 'convert' the heatmap into a more Gaussian-like heatmap. It ranks the pixels by intensity and takes the top- $k$  pixels, each becomes the centre of a Gaussian blob of size  $\sigma$  on the new heatmap. Some additional considerations are made for more robust training:

- **Sampling top- $k$  keypoints:** During experiments we choose  $k = 1000$  to capture as many keypoints in different areas as possible. As  $k$  gets large, the training speed might be slowed down. However a small value  $k$  tends to pick up neighbouring or nearby pixels rather than keypoints at different areas. Therefore we adapt a sampling approach: choose a random subset of  $k \cdot p$  keypoints in the top- $k$  pixels, where  $p = 0.1$ .
- **Gaussianised heatmap overlay:** We also adapt the Gaussianiser model as an overlay on the original heatmap, with a coefficient  $\lambda$ . It is also noticed that training with a higher  $\lambda$  incentives the model towards concentrating the heatmap around a few keypoints. Therefore we only use a small  $\lambda$  for the gaussianiser during training to allow the model to learn from mistakes in other non-top $k$  pixels in the original heatmap, and during inference we can set  $\lambda = 1$ .

The heatmap matching task is trained on FocalLoss and our self-defined loss function called the IntensityLoss. The IntensityLoss is defined as the F1Loss of the mean pixel intensity of the predicted heatmap and the ground-truth heatmap, which forces the model to concentrate the bright pixels around certain points rather than spreading the intensity throughout the whole heatmap.



Figure 6: Original heatmap generated from heatmap\_unet; Gaussianised heatmap with  $\lambda = 0.1$ ; Gaussianised heatmap with top- $k$  sampling and  $\lambda = 1$  (from left to right)



Figure 7: Gaussianised heatmap with  $\lambda = 0.5$  during training. The top- $k$  pixels are concentrated around a single keypoint.

The FocalLoss follows Lin et al’s [8] definition, with our chosen parameters as  $\alpha_1 = 0.99$  is the coefficient for the white-pixel class. and  $\gamma = 2$  as the power parameter for the  $(1 - \hat{y}_1)^\gamma$  term.

## 5.2. Inference procedure

The Gaussianised heatmap is then fed into a Bayesian Gaussian Mixture Model (BGMM) to extract the centroids which then become our keypoints. From the heatmap, we resample the pixels to collect a list of pixels  $(x, y)$  with a probability proportionate to the pixel’s intensity, which is then fed into the BGMM model. We set the `max_components=30`, then remove all predicted components whose weights are less than a predetermined `tolerance=0.01`. This ensures that components with very low weightage are removed from the final output, these components are usually covered by another component of a higher weightage. The keypoints are then transformed into world coordinates using the reverse process of the undistort-and-inverse-homography transformation described in Section 3.

A full inference pipeline is available in the inference test Jupyter Notebook at `experiments/ex06_player_heatmap_after_masking/spiideo-inference-test.ipynb`.

## 6. Experiments

All experiments are conducted on a Macbook M1 with local M1 MPS chips. The results of training the final model for competition is compiled in [this Wandb report](#).

Overall, the training results illustrate the model’s ability to capture the regions containing the players. The decreasing aggregate training and validation losses after 32 epochs indicate that the model can be further trained with more epochs and a bigger dataset to achieve better results.

Inspecting the model’s outputs gives us a hint on what the model struggles to predict. Two main problems arise through inspection:

- The model struggles to ‘find’ out players further to the camera than those nearer. Not only does the size of the players on the camera decreases significantly, the players may blend into the background, which either are cropped out by `masking_unet`, or are unable to be detected by `heatmap_unet`.



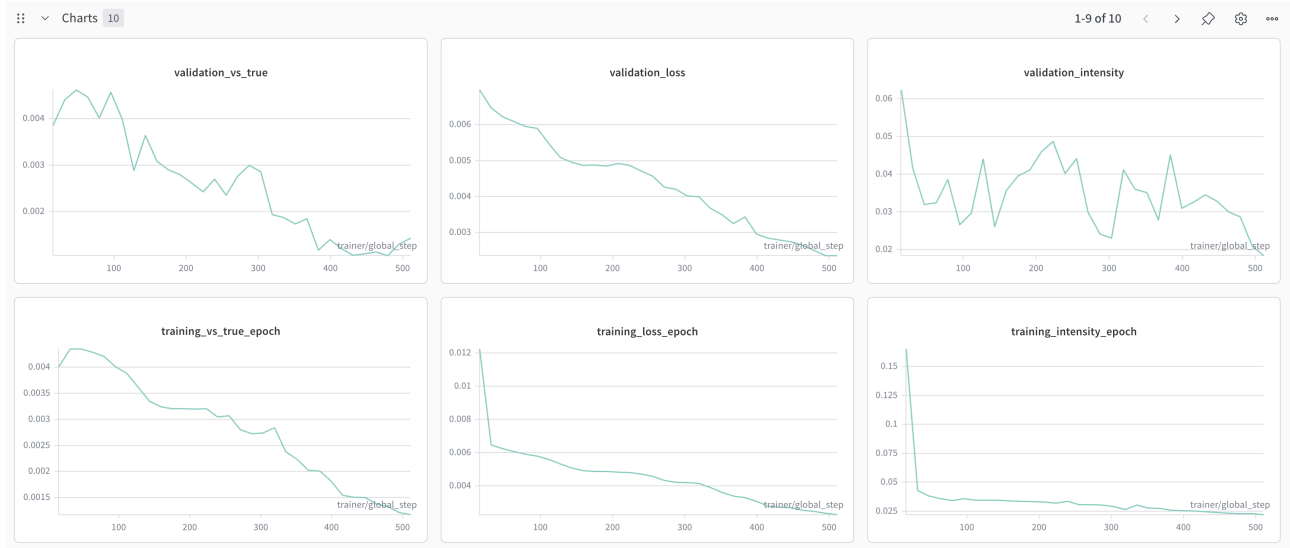


Figure 8: Loss charts after 32 epochs with a limited dataset of 256 images, resized at dimensions  $720 \times 405$ .

- The model tends to produce false positives, mostly on the field lines, the intersection of field lines and other foreign objects than players.



Figure 9: False positive pixels on the boundary between the field area and the spectators' seating area.

We also examine our diagnostics by running a full inference pipeline on all test and challenge images in the Spiideo SynLoc challenge. We take one image from the test set as an representative.

- The predictions verify our diagnosed problem during training that the model struggles to predict players further to the camera, whilst predicting false positives when encountering other objects such as field lines or goal posts.
- Some players are annotated twice in the prediction output. This is expected as BGMM (and other mixture-based approaches) fixes a number of components beforehand, and our component weight tolerance is set arbitrarily without consulting the dataset.

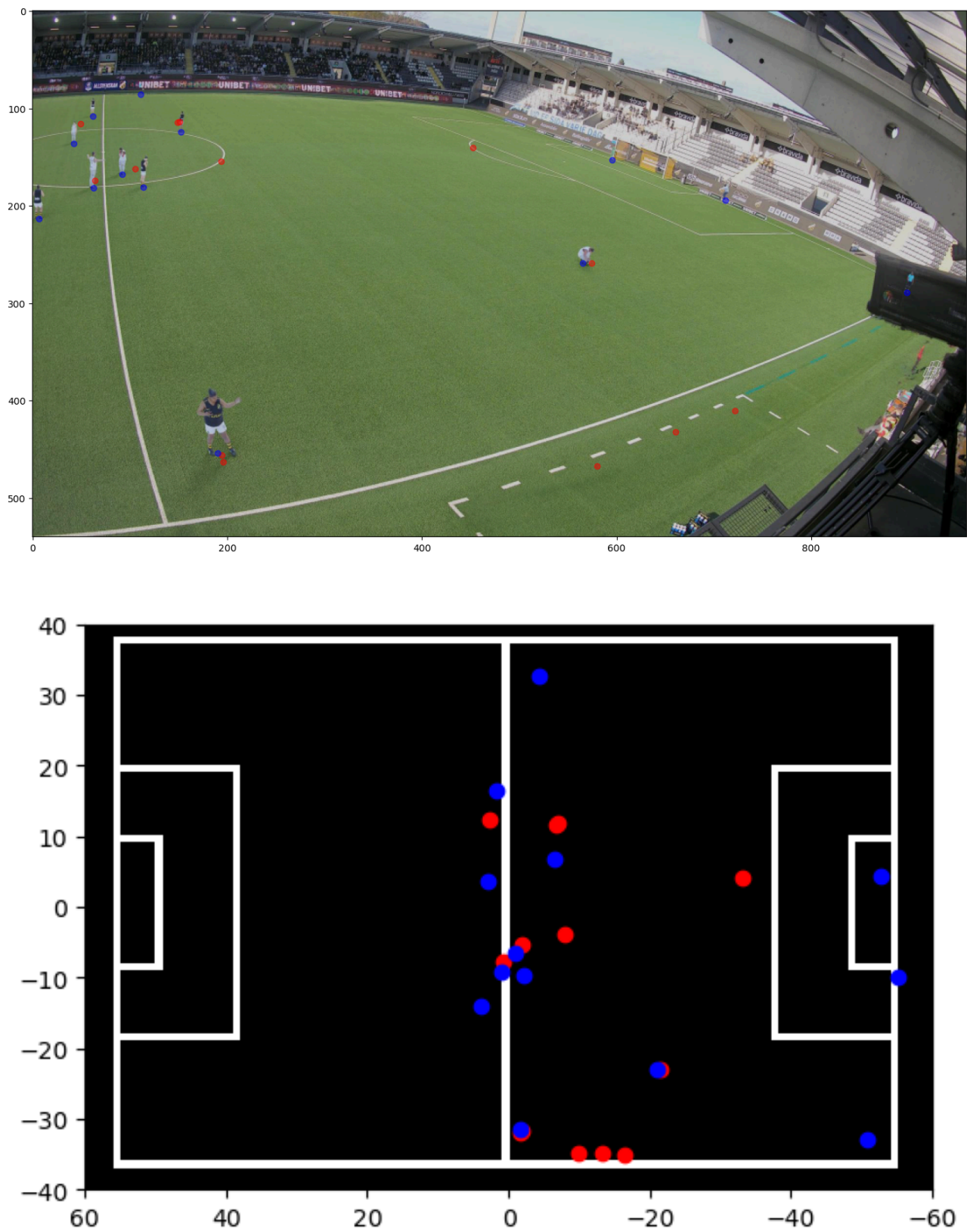


Figure 10: Diagnostic of prediction vs ground truth annotations on test image 000977 . jpg

## 7. Commentary and Future plans

The project has participated in the Spiideo’s SoccerNet SynLoc task on the test server, accessible on [CodaBench](https://codabench.com/competitions/soccer-net-synloc). At the point of submission (30 April, 2026), our project shows some progress on the

top-5 metrics but struggles to achieve positive results for the general metrics. More training would need to be devoted for the model, as well as more robust approaches are needed for the inference, non-trainable part of the pipeline.

Looking forward, the next immediate aim for the project is to migrate model training onto the cloud, through either the Lightning.AI platform [4] or renting High Performance Computes (HPCs) such as ASPIRE 2A, governed by the National Supercomputing Centre (NSCC), which is available for all students studying in autonomous universities in Singapore [11]. This will help the training phase become much faster than training on local processors such as GPUs or Apple's MPS chips, which will speed up the prototyping and batch-inference stages of development.

## Bibliography

- [1] N. S. Falaleev and R. Chen, "Enhancing Soccer Camera Calibration Through Keypoint Exploitation," Oct. 2024, pp. 65–73. doi: 10.1145/3689061.3689074.
- [2] H. Ardö *et al.*, "Spiideo SoccerNet SynLoc: Single Frame World Coordinate Athlete Detection and Localization with Synthetic Data;" in *Proceedings of the 20th International Joint Conference on Computer Vision, Imaging and Computer Graphics Theory and Applications*, Porto, Portugal: SCITEPRESS - Science, Technology Publications, 2025, pp. 278–285. doi: 10.5220/0013108200003912.
- [3] "ReCoDE-DeepLearning-Best-Practices." Accessed: Dec. 29, 2025. [Online]. Available: <https://imperialcollegelondon.github.io/ReCoDE-DeepLearning-Best-Practices/>
- [4] "Welcome to ⚡ PyTorch Lightning — PyTorch Lightning 2.6.1 documentation." Accessed: Apr. 29, 2026. [Online]. Available: <https://lightning.ai/docs/pytorch/stable/>
- [5] T.-Y. Lin *et al.*, "Microsoft COCO: Common Objects in Context." Accessed: Apr. 29, 2026. [Online]. Available: <https://arxiv.org/abs/1405.0312v3>
- [6] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation." Accessed: Nov. 03, 2025. [Online]. Available: <http://arxiv.org/abs/1505.04597>
- [7] A. Nibali, Z. He, S. Morgan, and L. Prendergast, "Numerical Coordinate Regression with Convolutional Neural Networks." Accessed: Apr. 06, 2026. [Online]. Available: <http://arxiv.org/abs/1801.07372>
- [8] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal Loss for Dense Object Detection." Accessed: Mar. 19, 2026. [Online]. Available: <http://arxiv.org/abs/1708.02002>
- [9] H. Qu, L. Xu, Y. Cai, L. G. Foo, and J. Liu, "Heatmap Distribution Matching for Human Pose Estimation."

- 
- [10] “SoccerNet - 2025.” Accessed: July 26, 2025. [Online]. Available: <https://www.soccer-net.org/challenges/2025>
- [11] W. C. Ding, “» Large-Scale Parallel Computations Using NSCC Supercomputer - NUS Information Technology | NUS IT Services, Solutions & Governance.” Accessed: Apr. 30, 2026. [Online]. Available: <https://nusit.nus.edu.sg/services/hpc-newsletter/large-scale-parallel-computations-using-nbcc-supercomputer/>
- 

## A. Project Github repository

<https://github.com/SamNguyen2k5/NUSC-ISC-FootballAnalysis/tree/isc-checkpoint-2>

## B. Participation in the SynLoc Spiideo Challenge, SoccerNet 2026

- Test server (including submission from @samnguyen157): <https://www.codabench.org/competitions/10128/#/results-tab>
- Challenge server: <https://www.codabench.org/competitions/10155/#/results-tab>

## C. Live reports on Wandb

<https://wandb.ai/samau07012005-national-university-of-singapore-students-/ISC-Football/reports/NUSC-ISC-2-Model-for-competition--VmlldzoxNjcxNzUwNA>